Ex92_RLS_linear_predictor

May 17, 2026

1 MSE StatDig : Chap 9 "Adaptive filtering"

1.1 Ex 9.2 Linear prediction using RLS on non stationary process

ver : DLY/18.05.2026

1.2 General

1.2.1 Description

The goal of this exercice is to compare the LMS and the RLS algorithm for a one-step prediction of a non-stationary process. Let $x[n]$ be a second-order autoregressive process AR(2) that is generated according to the difference equation:

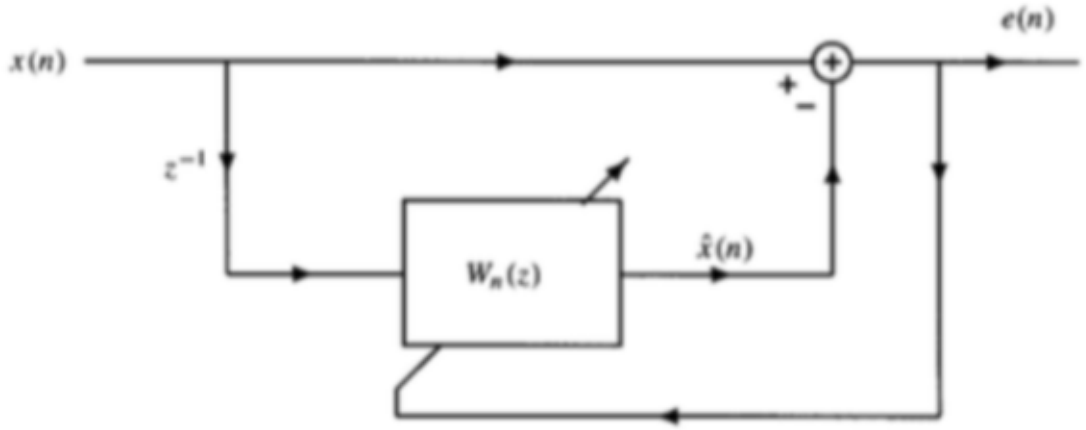
$$x[n] = -a_n[1] \cdot x[n-1] - a_n[2] \cdot x[n-2] + v[n]$$

Where the coefficients $a_n[k]$ are changing accross time:

$$\begin{cases} a_n[1] = -1.2728, a_n[2] = 0.81 \quad if \quad n \in [0, 99], \\ a_n[1] = 0, a_n[2] = 0.81 \quad if \quad n \in [100, 200], \end{cases}$$

Where $v[n]$ is a white Gaussian noise with variance $\sigma_v^2 = 1$.

To remind, the scheme of a one-step prediction is:



Therefore suppose we consider an adaptive linear predictor of the form:

$$\hat{x}[n] = w_n[1]x[n-1] + w_n[2]x[n-2]$$

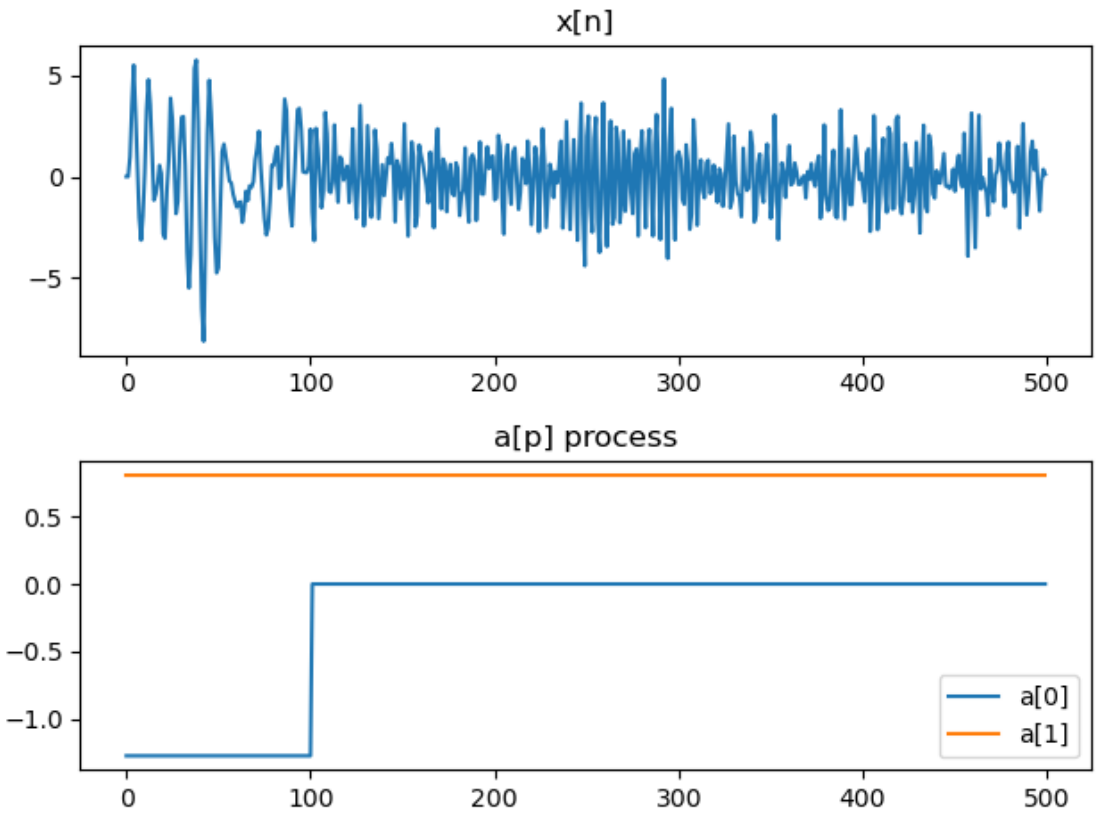
The goal is to design the prediction using the two adaptive algorithms (RLS and LMS) and to compare the performance.

1.2.2 Work

Ex1 : Signal generation

- Display the signals created by the following code as well the coefficients $a_n[k]$ used to generate the signal. :

```
N = 500
x = np.zeros(N)
A = np.zeros((2, N))

aFirst = np.array([-1.2728, 0.81])
aSecond = np.array([0, 0.81])

np.random.seed(0)
v = np.random.randn(N)

A[:, 0] = aFirst
A[:, 1] = aFirst

for n in range(2, N):
    if n <= 100:
        a = aFirst
    else:
        a = aSecond

    A[:, n] = a
    x[n] = -a[0] * x[n-1] - a[1] * x[n-2] + v[n]
```



Ex2 : Adaptive filter

Ex2.1 Recursive autocorrelation As seen in theory, the RLS algorithm can be expressed by computing the weighted deterministic autocorrelation matrix for x[n] and the deterministic cross-correlation between d[n] and x[n].

$$\mathbf{R}_x(n)\mathbf{w_n} = \mathbf{r}_{dx}(n)$$

$$\mathbf{w}_n = \mathbf{R}_x^{-1}(n) \cdot \mathbf{r}_{dx}(n)$$

The deterministic autocorrelation is computed:

$$\mathbf{R}_x(n) = \sum_{i=0}^{n} \lambda^{n-i} x[i]x^T[i] = \lambda \mathbf{R}_x(n-1) + \mathbf{x}^*[n]\mathbf{x}^T[n]$$

And the deterministic cross-correlation is computed:

$$\mathbf{r}_{dx}(n) = \sum_{i=0}^{n} \lambda^{n-i} d[i]\mathbf{x}^*[i] = \lambda \mathbf{r}_{dx}(n-1) + d[i]\mathbf{x}^T[i]$$

Where $x[i]$ is the sample vector which contains the last p samples:

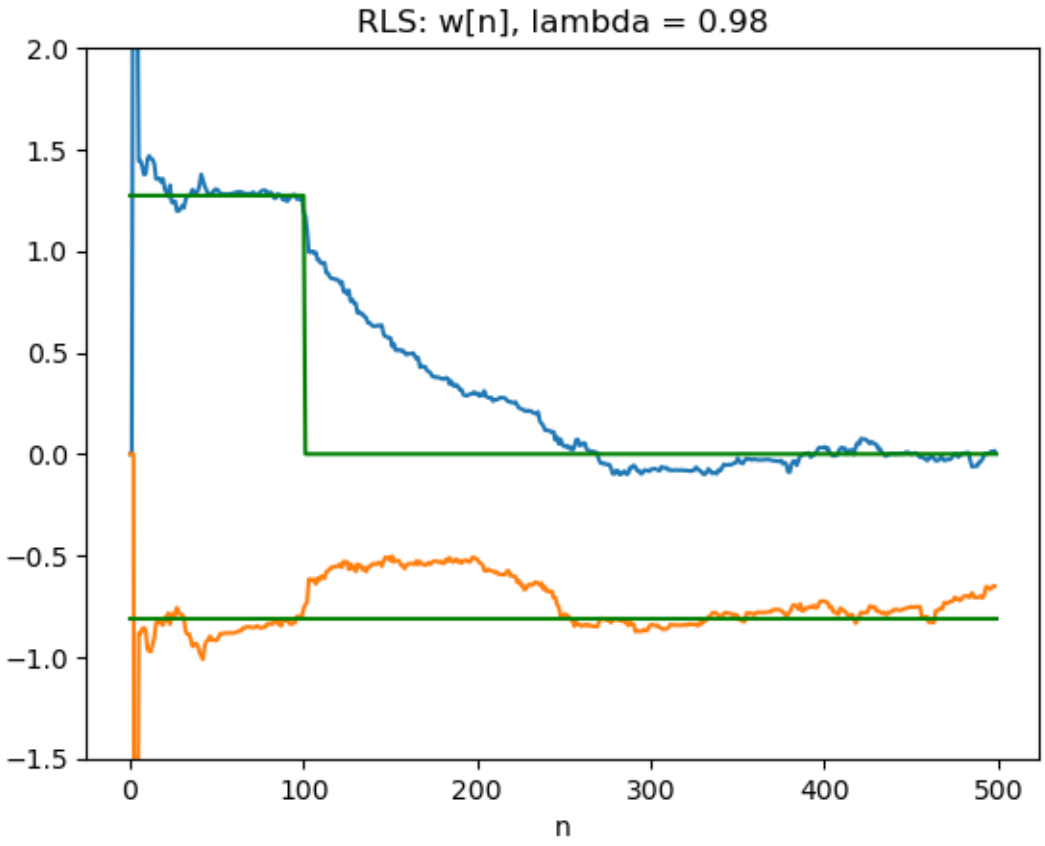
$$\mathbf{x}[i] = \begin{bmatrix} x[i] \\ x[i-1] \\ \vdots \\ x[i-p] \end{bmatrix}$$

And for a predictive filter, the desired output $d[i]$ is simply the future sample:

$$d[i] = x[i+1]$$

your work :

- Compute the deterministic autocorrelation matrix $\mathbf{R}_x(n)$ and the deterministic cross-correlation $\mathbf{r}_{dx}(n)$ using the above equations.
- Use the above equations to compute the filter coefficients $\mathbf{w}_n$ using the Wiener-Hopf equations.
- Compare the coefficients with the AR(2) coefficients.

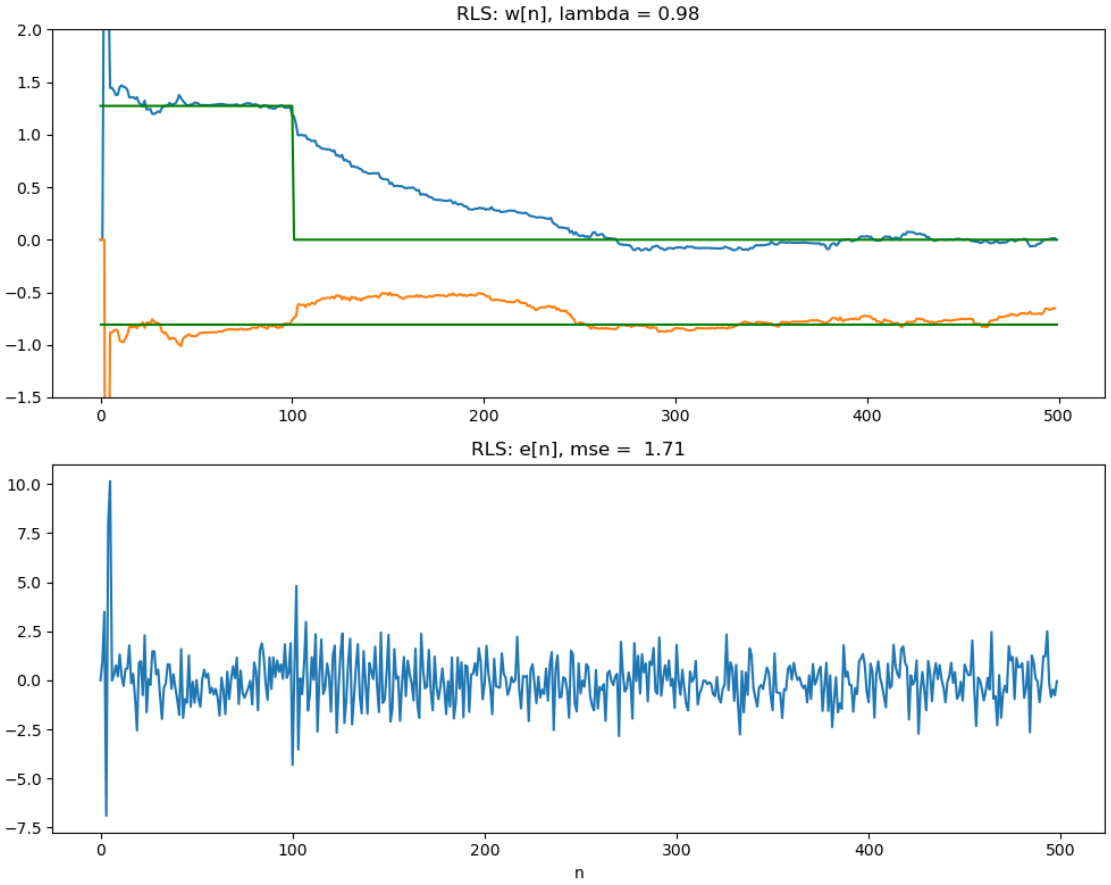


Ex2.2 Design RLS To avoid this inverse of $\mathbf{R}_x(n)$ accross time, we could compute the inverse recursively as well. For that, you can use the RLS calculation given in the theory and implement

the function `myRLS()`. Then, you can compare the found coefficients to the optimum coefficients. Use as previously p = 2 and $\lambda = 0.98$.

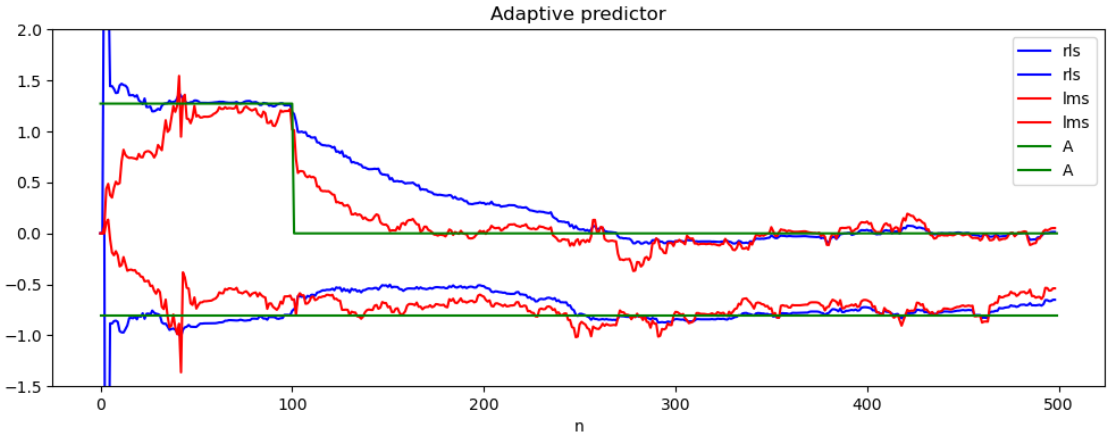
- Implement the function `myRLS()` and predict the signal x[n]

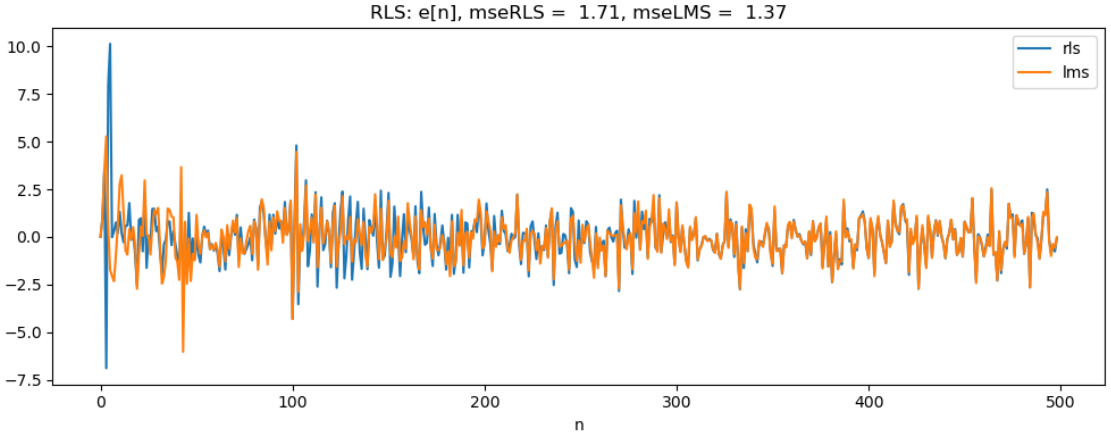
```
def myRLS(x, d, p, lambda_):
    """
    Recursive Least Squares (RLS) adaptive filter.

    Parameters:
        x : (N,) ndarray
            Input data to the adaptive filter (1D array).
        d : (N,) ndarray
            Desired output (1D array).
        p : int
            Number of filter coefficients.
        lambda_ : float
            Exponential forgetting factor (typically close to 1, e.g., 0.98).

    Returns:
        W : (N,p) ndarray
            Filter coefficient matrix (each row is the filter at time n).
        e : (N,) ndarray
            Error sequence over time.
    """

    return W,e
```



Ex2.3 Comparison with LMS

- Evaluate the prediction with the LMS algorithm by using your function `myLMS()`and the parameters $\mu = 0.02$.
- Plot the prediction and the error signal for both algorithms.


Adaptive predictor
rls
rls
lms
lms
A
A
n


RLS: e[n], mseRLS = 1.71, mseLMS = 1.37
rls
lms
n